

k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transferring the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

```
In [19]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [20]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [21]: # Run some setup code for this notebook.
from __future__ import print_function

import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
```

```
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```
In [22]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs682/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may ca
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

# Don't forget to run get_datasets.sh, or this will throw an error
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

Clear previously loaded data.

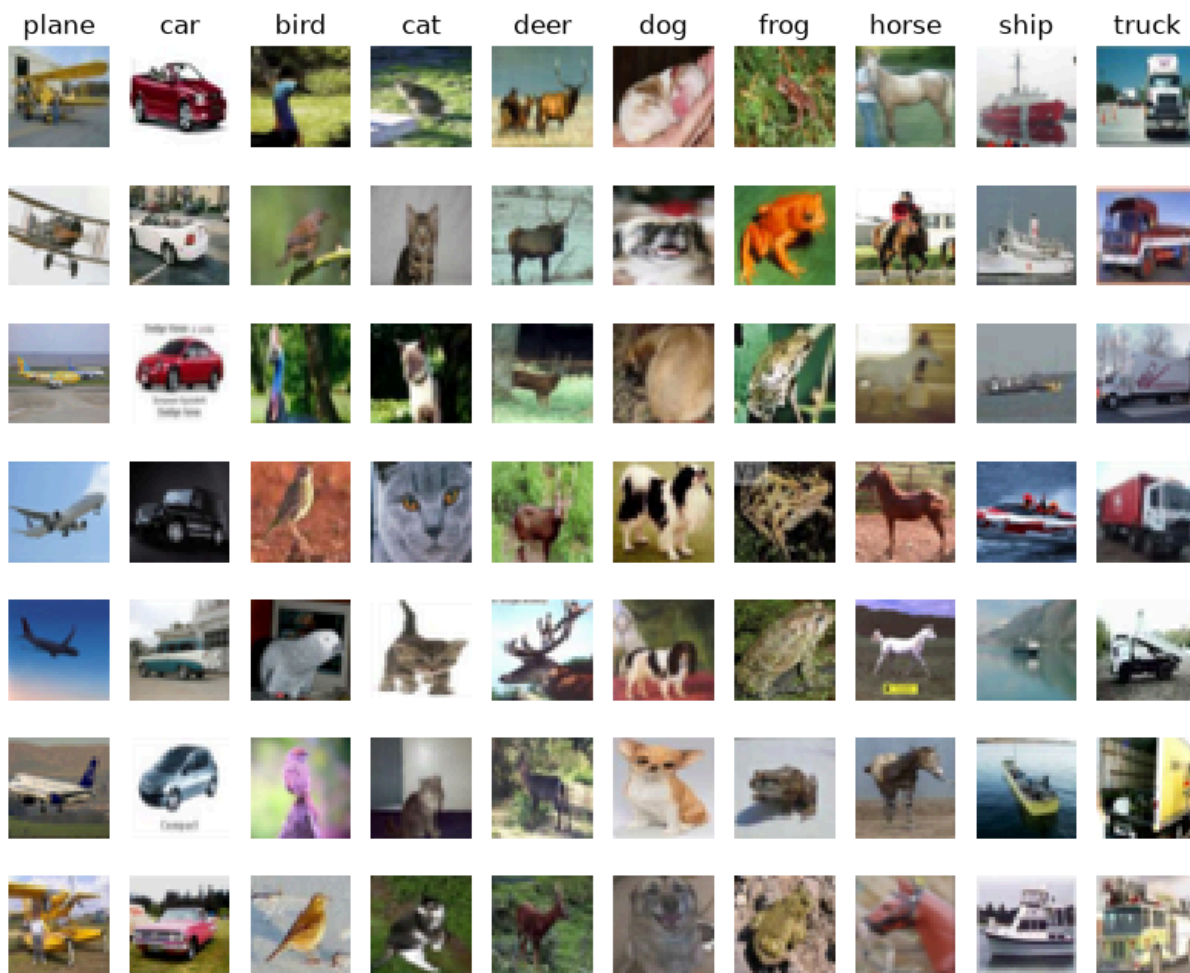
Training data shape: (50000, 32, 32, 3)

Training labels shape: (50000,)

Test data shape: (10000, 32, 32, 3)

Test labels shape: (10000,)

```
In [23]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 's
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
In [24]: # Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]
```

```
In [25]: # Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)

(5000, 3072) (500, 3072)
```

```
In [26]: from cs682.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are N_{tr} training examples and N_{te} test examples, this stage should result in a $N_{te} \times N_{tr}$ matrix where each element (i,j) is the distance between the i -th test and j -th train example.

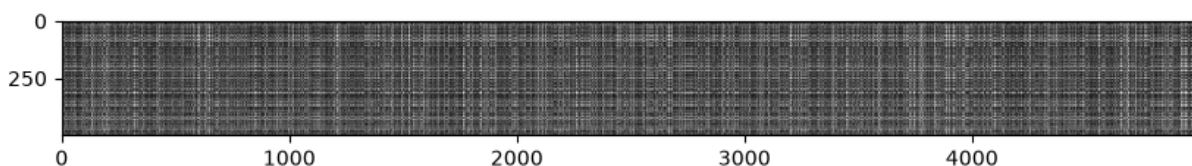
First, open `cs682/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

```
In [27]: # Open cs682/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

(500, 5000)

```
In [28]: # We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question #1: Notice the structured patterns in the distance matrix, where some rows or columns are visible brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Answer:

- **Bright rows:** a row is one *test* image. A bright row means that test image is far (in raw-pixel L2 distance) from almost every training image. This happens for test

images whose pixel statistics are unusual compared with the training set, e.g. very bright or very dark images, strong uniform backgrounds (white sky, black background), or unusual colors/high contrast. Because L2 on raw pixels is dominated by overall intensity and color, such images are far from everything.

- **Bright columns:** a column is one *training* image. A bright column means that training image is far from almost every test image, for the same reason: it is an atypical (very bright, very dark, or unusually colored) training example that no test image resembles at the pixel level.

```
In [29]: # Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 137 / 500 correct => accuracy: 0.274000

You should expect to see approximately 27% accuracy. Now let's try out a larger `k`, say `k = 5`:

```
In [30]: y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 139 / 500 correct => accuracy: 0.278000

You should expect to see a slightly better performance than with `k = 1`.

Inline Question #2: We can also use other distance metrics such as L1 distance. The performance of a Nearest Neighbor classifier that uses L1 distance will not change if (Select all that apply.):

1. The mean image of the training set is subtracted from the data.
2. All pixel values are divided by 255.
3. One mean is subtracted and the data is divided by one standard deviation, both computed over all pixel values.
4. Each color channel is standardized using its own mean and standard deviation (common in computer vision).
5. The coordinate axes for the data are rotated.
6. None of the above.

Answer: 1, 2, 3

Explanation: The L1 distance is $d(x, z) = \sum_p |x_p - z_p|$. A 1-NN (or k-NN) prediction only depends on the *ranking* of distances.

1. Subtracting the same mean image μ from every point:
 $|(x_p - \mu_p) - (z_p - \mu_p)| = |x_p - z_p|$, so all distances are unchanged.
2. Dividing every pixel by 255 scales every distance by the same factor $1/255$, so the ranking is unchanged.
3. Subtracting one scalar mean and dividing by one scalar std is a shift (cancels) followed by a uniform scale by $1/\sigma$, so the ranking is unchanged.
4. **Changes** performance: each channel is divided by a *different* σ_c , so the three channels are weighted differently in the sum and the ranking of neighbours can change.
5. **Changes** performance: L1 is not rotation invariant (unlike L2). E.g. $x = (1, 0)$ has L1 norm 1, but rotated by 45° it becomes $(1/\sqrt{2}, 1/\sqrt{2})$ with L1 norm $\sqrt{2}$, so relative distances and neighbours can change.

```
In [31]: # Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words, reduce
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('Difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

Difference was: 0.000000

Good! The distance matrices are the same

```
In [37]: # Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('Difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

Difference was: 0.000000

Good! The distance matrices are the same

```
In [33]: # Let's compare how fast the implementations are
def time_function(f, *args):
    """
```

```

    Call a function f with args and return the time (in seconds) that it took
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# you should see significantly faster performance with the fully vectorized

```

Two loop version took 13.567706 seconds

One loop version took 9.892429 seconds

No loop version took 0.327388 seconds

Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value $k = 5$ arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

```

In [34]: num_folds = 5
         k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

         X_train_folds = []
         y_train_folds = []
         #####
         # TODO:
         # Split up the training data into folds. After splitting, X_train_folds and
         # y_train_folds should each be lists of length num_folds, where
         # y_train_folds[i] is the label vector for the points in X_train_folds[i].
         # Hint: Look up the numpy array_split function.
         #####
         X_train_folds = np.array_split(X_train, num_folds)
         y_train_folds = np.array_split(y_train, num_folds)
         #####
         #
         #                                     END OF YOUR CODE
         #####

         # A dictionary holding the accuracies for different values of k that we find
         # when running cross-validation. After running cross-validation,
         # k_to_accuracies[k] should be a list of length num_folds giving the different
         # accuracy values that we found when using that value of k.
         k_to_accuracies = {}

         #####

```

```

# TODO:
# Perform k-fold cross validation to find the best value of k. For each
# possible value of k, run the k-nearest-neighbor algorithm num_folds times,
# where in each case you use all but one of the folds as training data and t
# last fold as a validation set. Store the accuracies for all fold and all
# values of k in the k_to_accuracies dictionary.
#####
for k in k_choices:
    k_to_accuracies[k] = []

for fold in range(num_folds):
    # fold `fold` is the validation set, the rest is the training set
    X_val_cv = X_train_folds[fold]
    y_val_cv = y_train_folds[fold]
    X_tr_cv = np.concatenate([X_train_folds[j] for j in range(num_folds) if
    y_tr_cv = np.concatenate([y_train_folds[j] for j in range(num_folds) if

    cv_classifier = KNearestNeighbor()
    cv_classifier.train(X_tr_cv, y_tr_cv)
    # distances do not depend on k, so compute them once per fold
    cv_dists = cv_classifier.compute_distances_no_loops(X_val_cv)

    for k in k_choices:
        y_val_pred = cv_classifier.predict_labels(cv_dists, k=k)
        accuracy = np.mean(y_val_pred == y_val_cv)
        k_to_accuracies[k].append(accuracy)
#####
#                                     END OF YOUR CODE
#####

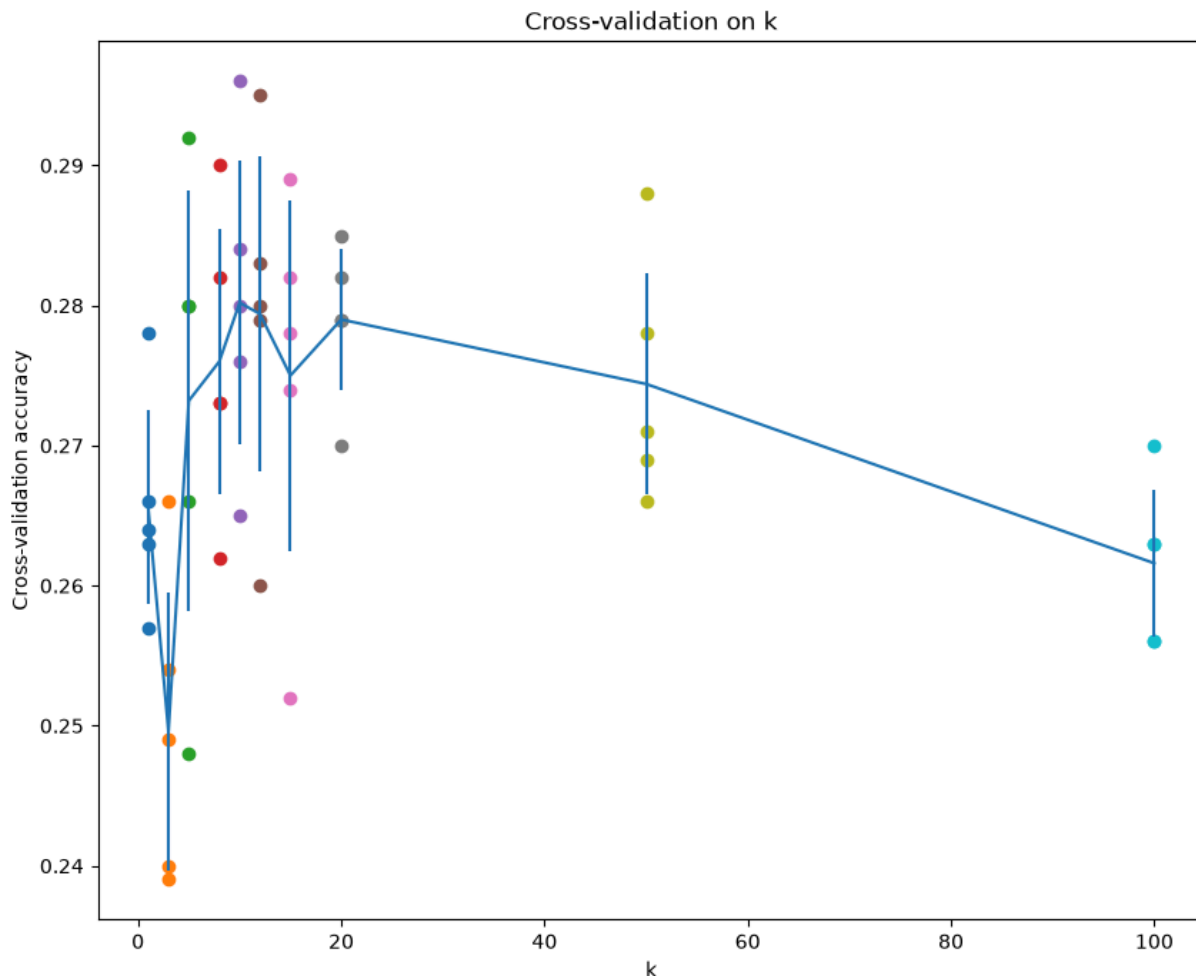
# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))

```

```
k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.263000
```

```
In [35]: # plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)
```

```
# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.items)])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.items)])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()
```



```
In [36]: # Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = 10
```

```
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)
```

```
# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 141 / 500 correct => accuracy: 0.282000

Inline Question 3 Which of the following statements about k -Nearest Neighbor (k -NN) are true in a classification setting, and for all k ? Select all that apply.

1. The training error of a 1-NN will always be better than or equal to that of 5-NN.
2. The test error of a 1-NN will always be better than that of a 5-NN.
3. The decision boundary of the k -NN classifier is linear.
4. The time needed to classify a test example with the k -NN classifier grows with the size of the training set.
5. None of the above.

Answer: 1, 4

Explanation:

1. **True.** With 1-NN every training point is its own nearest neighbour (distance 0), so the training error is 0 (assuming no duplicate images with different labels). 5-NN can only have training error ≥ 0 , so 1-NN is always better than or equal.
2. **False.** 1-NN overfits to noise in the training set; a larger k often generalises better (in this notebook $k = 5$ and the cross-validated k beat $k = 1$ on test data). It is not "always" either way.
3. **False.** The k -NN decision boundary is formed from pieces of the perpendicular bisectors between training points (a Voronoi-style partition); it is piecewise linear and in general highly non-linear, not a single linear boundary.
4. **True.** k -NN has no real training step; to classify one test point it must compute the distance to every training point, so the cost grows linearly with the training set size, $O(N \cdot D)$.

Multiclass Support Vector Machine exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the SVM
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
In [64]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [65]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK FIRST ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [66]: # Run some setup code for this notebook.
from __future__ import print_function
import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipy
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:
`%reload_ext autoreload`

CIFAR-10 Data Loading and Preprocessing

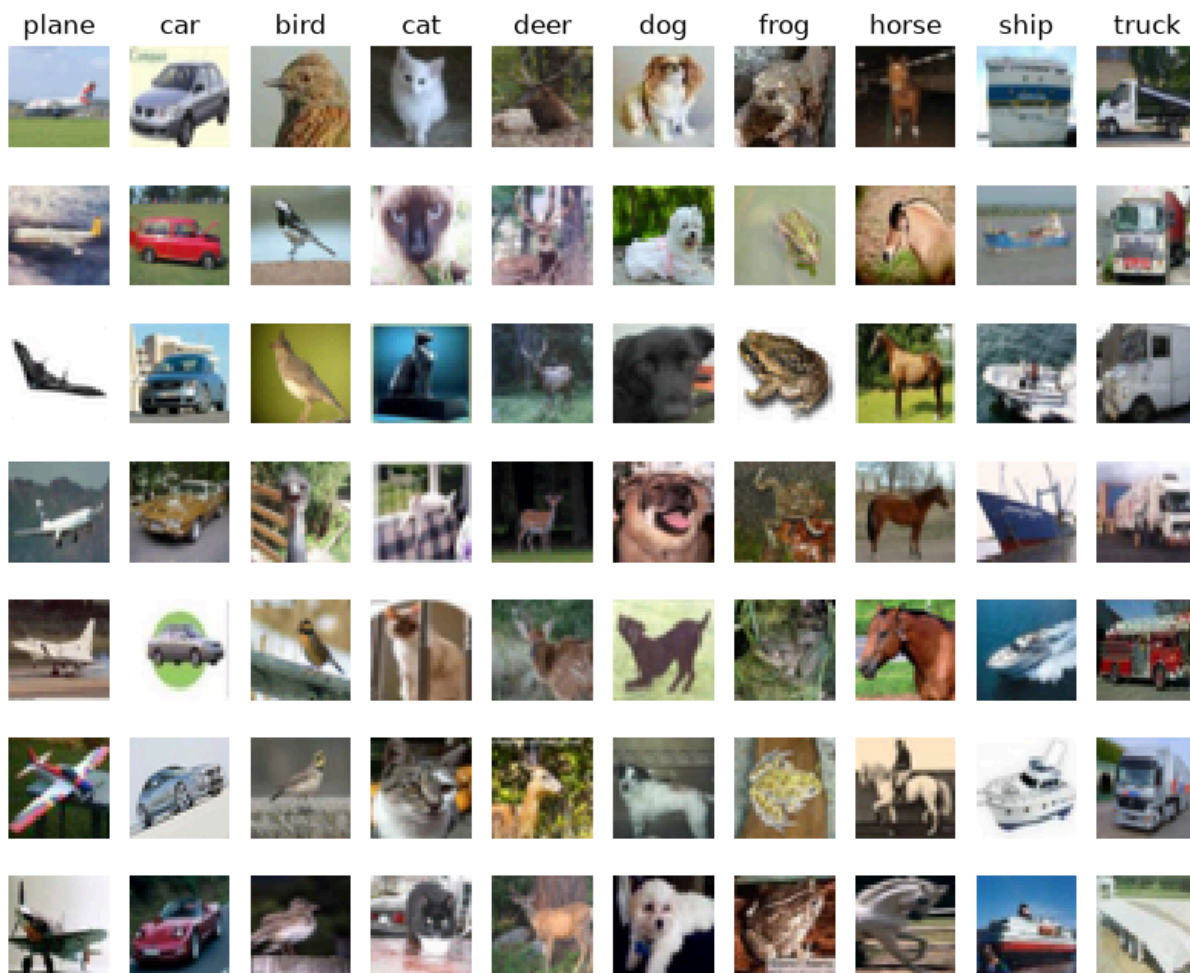
```
In [67]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs682/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may ca
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass
# Don't forget to run get_datasets.sh, or this will throw an error
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Clear previously loaded data.
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape: (10000, 32, 32, 3)
Test labels shape: (10000,)
```

```
In [68]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 's
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
In [69]: # Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]
```

```

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

```

```

Train data shape: (49000, 32, 32, 3)
Train labels shape: (49000,)
Validation data shape: (1000, 32, 32, 3)
Validation labels shape: (1000,)
Test data shape: (1000, 32, 32, 3)
Test labels shape: (1000,)

```

```

In [70]: # Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)

```

```

Training data shape: (49000, 3072)
Validation data shape: (1000, 3072)
Test data shape: (1000, 3072)
dev data shape: (500, 3072)

```

```

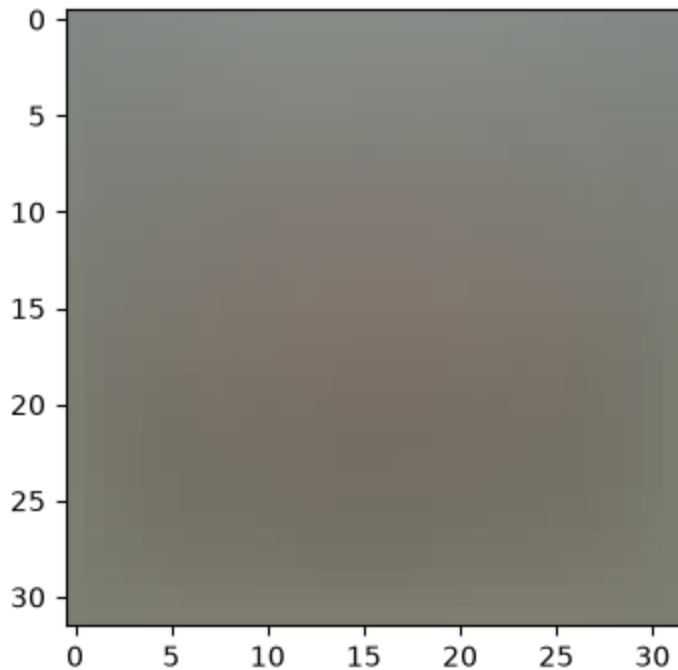
In [71]: # Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean image
plt.show()

```

```

[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]

```



```
In [72]: # second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image
```

```
In [73]: # third: append the bias dimension of ones (i.e. bias trick) so that our SVM
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)

(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

SVM Classifier

Your code for this section will all be written inside **cs682/classifiers/linear_svm.py**.

As you can see, we have prefilled the function `svm_loss_naive` which uses for loops to evaluate the multiclass SVM loss function.

```
In [74]: # Evaluate the naive implementation of the loss we provided for you:
from cs682.classifiers.linear_svm import svm_loss_naive
import time

# generate a random SVM weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001
```

```
loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))
```

loss: 9.149796

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the SVM cost function and implement it inline inside the function `svm_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient correctly, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

```
In [75]: # Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions,
# compare them with your analytically computed gradient. The numbers should
# almost exactly align along all dimensions.
from cs682.gradient_check import grad_check_sparse
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = svm_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: 4.597325 analytic: 4.597325, relative error: 2.240111e-11
numerical: -7.762665 analytic: -7.762665, relative error: 1.065460e-11
numerical: 17.781522 analytic: 17.781522, relative error: 1.096751e-11
numerical: 12.290537 analytic: 12.290537, relative error: 2.989880e-11
numerical: -14.311770 analytic: -14.311770, relative error: 7.345401e-12
numerical: -19.019036 analytic: -19.019036, relative error: 4.140373e-12
numerical: -8.931383 analytic: -8.931383, relative error: 3.421086e-11
numerical: 34.412771 analytic: 34.412771, relative error: 8.254929e-13
numerical: 6.382881 analytic: 6.382881, relative error: 2.571820e-11
numerical: -1.213852 analytic: -1.214176, relative error: 1.332281e-04
numerical: 4.588690 analytic: 4.588690, relative error: 1.405596e-10
numerical: 2.754356 analytic: 2.754356, relative error: 1.181884e-11
numerical: -18.333518 analytic: -18.333518, relative error: 1.339114e-11
numerical: -19.898355 analytic: -19.898355, relative error: 1.707586e-12
numerical: -4.172778 analytic: -4.172778, relative error: 3.382207e-11
numerical: -19.779520 analytic: -19.779520, relative error: 9.516133e-12
numerical: 3.882263 analytic: 3.882263, relative error: 3.579372e-11
numerical: 22.230817 analytic: 22.211512, relative error: 4.343833e-04
numerical: -14.674736 analytic: -14.674736, relative error: 8.328328e-12
numerical: 4.813613 analytic: 4.813613, relative error: 4.219777e-11
```

Inline Question 1:

It is possible that once in a while a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a gradient check could fail? How would change the margin affect of the frequency of this happening? *Hint: the SVM loss function is not strictly speaking differentiable*

Answer: With scores $s_j = W_j \cdot x_i$, the hinge loss is

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

and its gradient uses the indicator $1(J) = 1$ if class j violates the margin ($s_j - s_{y_i} + 1 > 0$ and $j \neq y_i$), 0 otherwise:

$$\frac{\partial L_i}{\partial W_j} = 1(J) x_i, \quad \frac{\partial L_i}{\partial W_{y_i}} = - \sum_{j \neq y_i} 1(J) x_i$$

Cause of the mismatch: $\max(0, \cdot)$ has a kink where $s_j - s_{y_i} + 1 = 0$, so the loss is not differentiable there and $1(J)$ jumps from 0 to 1. The numerical gradient uses a centered difference $\frac{L(W+h) - L(W-h)}{2h}$. If a margin is within h of the kink, the two evaluation points fall on opposite sides, so the numerical estimate averages the two slopes while the analytic gradient picks one side.

Is it a concern? No. It is an artifact of checking the gradient numerically at a non-differentiable point, not a bug in the analytic gradient. It affects only an occasional dimension, and gradient descent works fine with this subgradient.

1-D example: $f(x) = \max(0, x)$ at $x = 0$. The analytic gradient we use is 0 (with the indicator convention $1 = 0$ when the margin is not > 0), but the numerical gradient is $\frac{f(h) - f(-h)}{2h} = \frac{h - 0}{2h} = 0.5$.

Effect of the margin: Call the margin Δ (it is 1 in the lecture formula). At initialization W is small, so all $s \approx 0$ and every term is $s_j - s_{y_i} + \Delta \approx \Delta = 1$, far from the kink, so failures are rare. Making Δ smaller (toward 0) puts many terms right at the kink and makes mismatches much more frequent; a larger Δ makes them rarer. In general the frequency depends on how many margin terms land within h of 0.

```
In [76]: # Next implement the function svm_loss_vectorized; for now only compute the
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))
```

```

from cs682.classifiers.linear_svm import svm_loss_vectorized
tic = time.time()
loss_vectorized, _ = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much
print('difference: %f' % (loss_naive - loss_vectorized))

```

Naive loss: 9.149796e+00 computed in 0.056214s
 Vectorized loss: 9.149796e+00 computed in 0.006264s
 difference: -0.000000

```

In [77]: # Complete the implementation of svm_loss_vectorized, and compute the gradient
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)

```

Naive loss and gradient: computed in 0.058421s
 Vectorized loss and gradient: computed in 0.002161s
 difference: 0.000000

Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss.

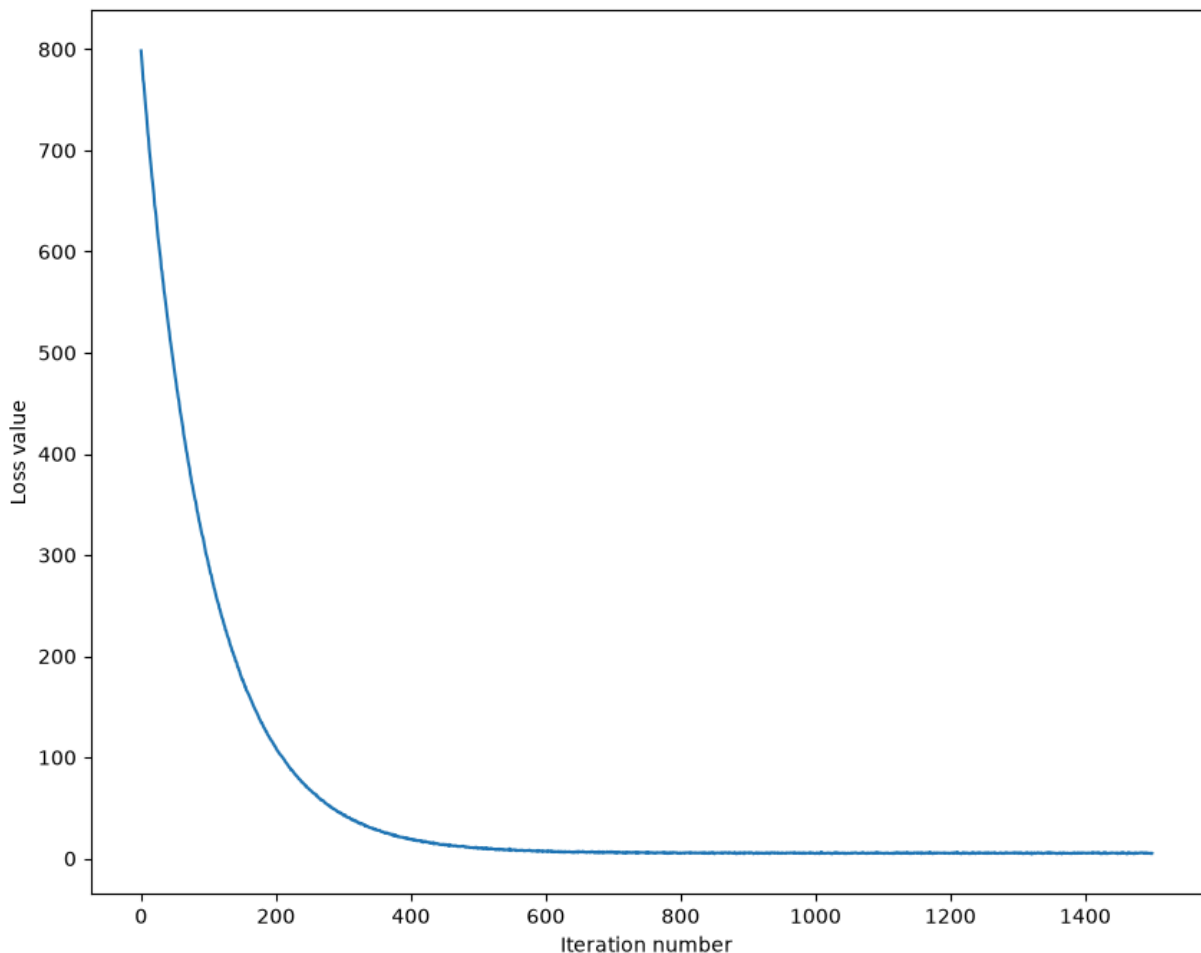
```

In [78]: # In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs682.classifiers import LinearSVM
svm = LinearSVM()
tic = time.time()
loss_hist = svm.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                      num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))

```

```
iteration 0 / 1500: loss 798.548101
iteration 100 / 1500: loss 290.237814
iteration 200 / 1500: loss 109.144227
iteration 300 / 1500: loss 42.514514
iteration 400 / 1500: loss 18.795323
iteration 500 / 1500: loss 9.899851
iteration 600 / 1500: loss 7.320198
iteration 700 / 1500: loss 6.217382
iteration 800 / 1500: loss 4.998459
iteration 900 / 1500: loss 5.601807
iteration 1000 / 1500: loss 4.970029
iteration 1100 / 1500: loss 5.215555
iteration 1200 / 1500: loss 5.384175
iteration 1300 / 1500: loss 5.496114
iteration 1400 / 1500: loss 5.328431
That took 2.376542s
```

```
In [79]: # A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



```
In [80]: # Write the LinearSVM.predict function and evaluate the performance on both
# training and validation set
y_train_pred = svm.predict(X_train)
```

```
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = svm.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

training accuracy: 0.370898
validation accuracy: 0.387000

```
In [81]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able
# to get a classification accuracy of about 0.4 on the validation set.
learning_rates = [1e-7, 5e-5]
regularization_strengths = [2.5e4, 5e4]

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1 # The highest validation accuracy that we have seen so far.
best_svm = None # The LinearSVM object that achieved the highest validation

#####
# TODO:
# Write code that chooses the best hyperparameters by tuning on the validation
# set. For each combination of hyperparameters, train a linear SVM on the
# training set, compute its accuracy on the training and validation sets, and
# store these numbers in the results dictionary. In addition, store the best
# validation accuracy in best_val and the LinearSVM object that achieves this
# accuracy in best_svm.
#
# Hint: You should use a small value for num_iters as you develop your
# validation code so that the SVMs don't take much time to train; once you are
# confident that your validation code works, you should rerun the validation
# code with a larger value for num_iters.
#####
# Extend the provided values with a few more around 1e-7. Pairs with
# lr * reg >= 1 are skipped: the regularization step then overshoots and
# training diverges (this is the case for lr = 5e-5).
search_lrs = learning_rates + [2e-7, 5e-7]
search_regs = regularization_strengths + [1e4]

for lr in search_lrs:
    for reg in search_regs:
        if lr * reg >= 1:
            continue
        svm = LinearSVM()
        svm.train(X_train, y_train, learning_rate=lr, reg=reg,
                  num_iters=1500, verbose=False)
        train_accuracy = np.mean(svm.predict(X_train) == y_train)
        val_accuracy = np.mean(svm.predict(X_val) == y_val)
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_svm = svm
#####
```

```
#
# END OF YOUR CODE
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' % best)

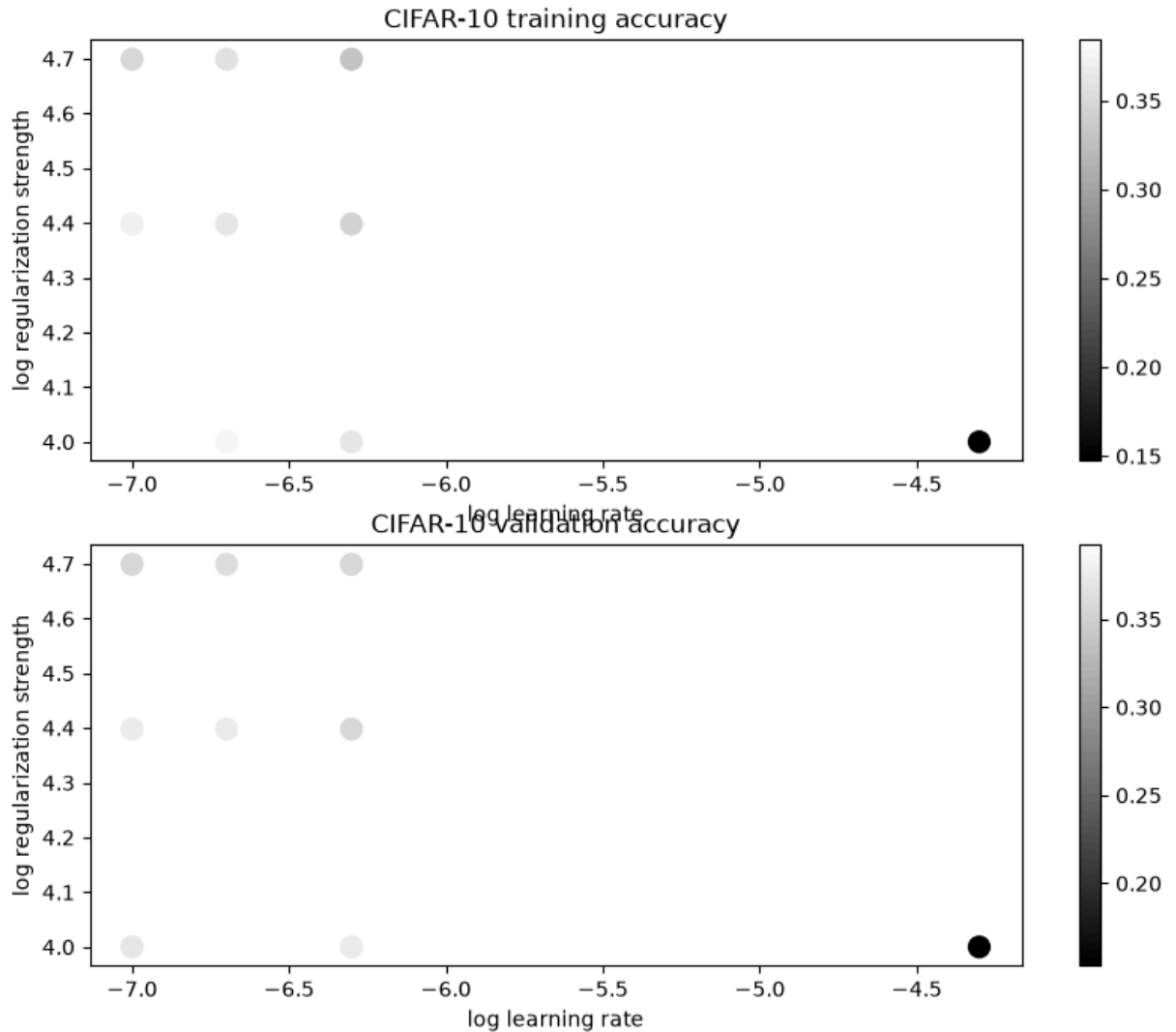
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.384163 val accuracy: 0.37
2000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.372245 val accuracy: 0.37
5000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.348755 val accuracy: 0.35
5000
lr 2.000000e-07 reg 1.000000e+04 train accuracy: 0.376918 val accuracy: 0.39
2000
lr 2.000000e-07 reg 2.500000e+04 train accuracy: 0.361939 val accuracy: 0.37
6000
lr 2.000000e-07 reg 5.000000e+04 train accuracy: 0.356857 val accuracy: 0.36
3000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.361286 val accuracy: 0.37
5000
lr 5.000000e-07 reg 2.500000e+04 train accuracy: 0.343776 val accuracy: 0.35
7000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.328265 val accuracy: 0.35
4000
lr 5.000000e-05 reg 1.000000e+04 train accuracy: 0.147224 val accuracy: 0.15
3000
best validation accuracy achieved during cross-validation: 0.392000
```

```
In [82]: # Visualize the cross-validation results
import math
x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
```

```
plt.title('CIFAR-10 validation accuracy')
plt.show()
```



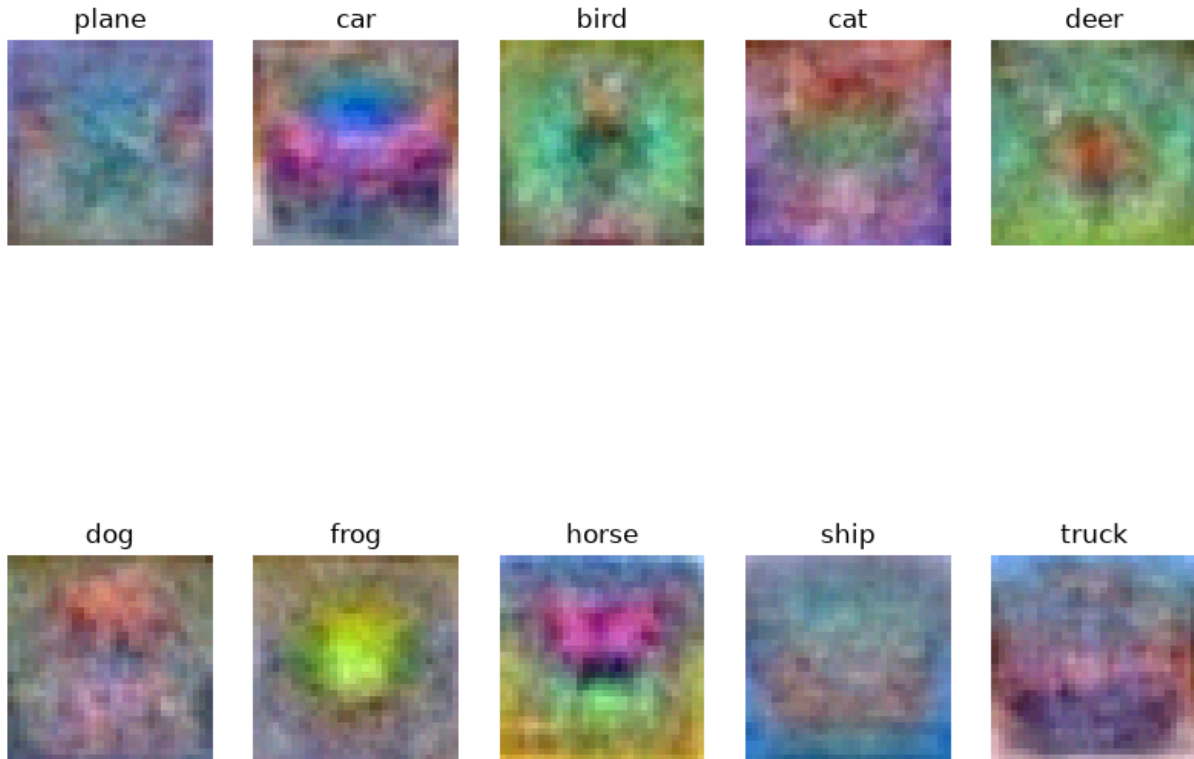
```
In [83]: # Evaluate the best svm on test set
y_test_pred = best_svm.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('linear SVM on raw pixels final test set accuracy: %f' % test_accuracy)
```

linear SVM on raw pixels final test set accuracy: 0.362000

```
In [84]: # Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, the
# or may not be nice to look at.
w = best_svm.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 's
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
```

```
plt.axis('off')
plt.title(classes[i])
```



Inline question 2:

Describe what your visualized SVM weights look like, and offer a brief explanation for why they look the way that they do.

Answer: The weights look like blurry, grainy templates of each class, dominated by color. The clearest ones: *car* has a red body with a blue area on top (windshield/sky), *frog* is a bright green blob in the center, *deer* has a brown center on a green background, *horse* has a dark and pinkish shape in the middle above a yellow-brown bottom, *truck* has a red/purple lower half, and *plane* and *ship* are bluish overall, with *ship* darker blue at the bottom (water). *Bird*, *cat* and *dog* are fainter reddish-brown or greenish blobs.

Why: A linear classifier $f(x, W) = Wx$ scores class j as $s_j = W_j \cdot x_i$, so each W_j acts as a template matched against the image. Training pushes W_j toward the pixel patterns common in class j , mostly overall color and background layout, which is why color blobs dominate. Each class gets only one template, so all poses, colors and left/right orientations in a class are averaged together, which makes the templates blurry. They stay grainy because the best model used the weakest regularization I tried ($\lambda = 1e4$, with $lr = 2e-7$) and only 1500 SGD iterations: weak L2 regularization $\lambda R(W) = \lambda \sum w^2$ does little to suppress weights on individual noisy pixels. Stronger regularization smooths the templates but lowered validation accuracy in my sweep (0.376 at $\lambda = 2.5e4$ and 0.363 at $\lambda = 5e4$, vs 0.392 at $1e4$, for $lr = 2e-7$)

Softmax exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

This exercise is analogous to the SVM exercise. You will:

- implement a fully-vectorized **loss function** for the Softmax classifier
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** with numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
In [11]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [12]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK FIRST ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [13]: from __future__ import print_function
import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipy
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:
 %reload_ext autoreload

```

In [14]: def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000,
    """
    Load the CIFAR-10 dataset from disk and perform preprocessing to prepare
    it for the linear classifier. These are the same steps as we used for the
    SVM, but condensed to a single function.
    """
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs682/datasets/cifar-10-batches-py'
    # Don't forget to run get_datasets.sh, or this will throw an error
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]
    mask = np.random.choice(num_training, num_dev, replace=False)
    X_dev = X_train[mask]
    y_dev = y_train[mask]

    # Preprocessing: reshape the image data into rows
    X_train = np.reshape(X_train, (X_train.shape[0], -1))
    X_val = np.reshape(X_val, (X_val.shape[0], -1))
    X_test = np.reshape(X_test, (X_test.shape[0], -1))
    X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

    # Normalize the data: subtract the mean image
    mean_image = np.mean(X_train, axis = 0)
    X_train -= mean_image
    X_val -= mean_image
    X_test -= mean_image
    X_dev -= mean_image

    # add bias dimension and transform into columns
    X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
    X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
    X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
    X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

    return X_train, y_train, X_val, y_val, X_test, y_test, X_dev, y_dev

# Cleaning up variables to prevent loading data multiple times (which may cause
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

```

```
# Invoke the above function to get our data.
X_train, y_train, X_val, y_val, X_test, y_test, X_dev, y_dev = get_CIFAR10_data()
print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
print('dev data shape: ', X_dev.shape)
print('dev labels shape: ', y_dev.shape)
```

Clear previously loaded data.

Train data shape: (49000, 3073)

Train labels shape: (49000,)

Validation data shape: (1000, 3073)

Validation labels shape: (1000,)

Test data shape: (1000, 3073)

Test labels shape: (1000,)

dev data shape: (500, 3073)

dev labels shape: (500,)

Softmax Classifier

Your code for this section will all be written inside **cs682/classifiers/softmax.py**.

```
In [15]: # First implement the naive softmax loss function with nested loops.
# Open the file cs682/classifiers/softmax.py and implement the
# softmax_loss_naive function.

from cs682.classifiers.softmax import softmax_loss_naive
import time

# Generate a random softmax weight matrix and use it to compute the loss.
W = np.random.randn(3073, 10) * 0.0001
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

loss: 2.421284

sanity check: 2.302585

Inline Question 1:

Why do we expect our loss to be close to $-\log(0.1)$? Explain briefly.**

Your answer: In the lecture notation the scores are $z_j = W_j x_i$, the softmax gives

$$p(y = i | x) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \text{ and the loss is } L_i = -\log\left(\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}\right).$$

W is initialised with very small random values ($\times 0.0001$), so every $z_j \approx 0$. Then each $e^{z_j} \approx 1$ and, with $K = 10$ classes, $p(y = y_i | x) \approx \frac{1}{10} = 0.1$. Therefore $L_i \approx -\log(0.1) = \log 10 \approx 2.303$, and the average $L = \frac{1}{N} \sum_i L_i$ (no regularization here) is the same. This is a sanity check that the loss is implemented correctly.

```
In [16]: # Complete the implementation of softmax_loss_naive and implement a (naive)
# version of the gradient that uses nested loops.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# As we did for the SVM, use numeric gradient checking as a debugging tool.
# The numeric gradient should be close to the analytic gradient.
from cs682.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad, 10)

# similar to SVM case, do another gradient check with regularization
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad, 10)
```

```
numerical: 0.266113 analytic: 0.266113, relative error: 4.079215e-09
numerical: 0.681800 analytic: 0.681800, relative error: 7.974048e-08
numerical: -1.648941 analytic: -1.648942, relative error: 1.269473e-08
numerical: -2.280319 analytic: -2.280319, relative error: 4.440655e-09
numerical: -1.839154 analytic: -1.839154, relative error: 2.194938e-08
numerical: -0.242307 analytic: -0.242307, relative error: 2.612713e-07
numerical: -2.911632 analytic: -2.911632, relative error: 3.675620e-09
numerical: -0.635279 analytic: -0.635279, relative error: 1.256604e-07
numerical: 1.231575 analytic: 1.231575, relative error: 1.280162e-08
numerical: -4.010223 analytic: -4.010223, relative error: 4.999984e-09
numerical: 3.438571 analytic: 3.438571, relative error: 1.813460e-08
numerical: -0.165316 analytic: -0.165316, relative error: 6.082173e-08
numerical: 0.685930 analytic: 0.685930, relative error: 3.243538e-08
numerical: -0.408935 analytic: -0.408935, relative error: 1.069412e-08
numerical: -2.012154 analytic: -2.012154, relative error: 1.479997e-08
numerical: 0.942949 analytic: 0.942949, relative error: 2.542433e-08
numerical: 2.716527 analytic: 2.716527, relative error: 7.432499e-09
numerical: 5.152193 analytic: 5.152193, relative error: 1.083130e-08
numerical: -1.137575 analytic: -1.137575, relative error: 5.284023e-08
numerical: 2.285663 analytic: 2.285662, relative error: 4.589834e-08
```

```
In [17]: # Now that we have a naive implementation of the softmax loss function and i
# implement a vectorized version in softmax_loss_vectorized.
# The two versions should compute the same results, but the vectorized versi
# much faster.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from cs682.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev,
toc = time.time()
```

```

print('vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# As we did for the SVM, we use the Frobenius norm to compare the two versions
# of the gradient.
grad_difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('Loss difference: %f' % np.abs(loss_naive - loss_vectorized))
print('Gradient difference: %f' % grad_difference)

```

naive loss: 2.421284e+00 computed in 0.055501s
vectorized loss: 2.421284e+00 computed in 0.013152s
Loss difference: 0.000000
Gradient difference: 0.000000

```

In [18]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able
# to get a classification accuracy of over 0.35 on the validation set.
from cs682.classifiers import Softmax
results = {}
best_val = -1
best_softmax = None
learning_rates = [1e-7, 5e-7]
regularization_strengths = [2.5e4, 5e4]

#####
# TODO:
# Use the validation set to set the learning rate and regularization strength.
# This should be identical to the validation that you did for the SVM; save
# the best trained softmax classifier in best_softmax.
#####
# Extend the provided values with one more learning rate and a weaker
# regularization strength.
search_lrs = learning_rates + [2e-7]
search_regs = regularization_strengths + [1e4]

for lr in search_lrs:
    for reg in search_regs:
        softmax = Softmax()
        softmax.train(X_train, y_train, learning_rate=lr, reg=reg,
                      num_iters=1500, verbose=False)
        train_accuracy = np.mean(softmax.predict(X_train) == y_train)
        val_accuracy = np.mean(softmax.predict(X_val) == y_val)
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax

#####
#                                     END OF YOUR CODE
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

```

```
print('best validation accuracy achieved during cross-validation: %f' % best
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.353061 val accuracy: 0.36
5000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.329347 val accuracy: 0.34
6000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.306816 val accuracy: 0.32
0000
lr 2.000000e-07 reg 1.000000e+04 train accuracy: 0.359286 val accuracy: 0.37
2000
lr 2.000000e-07 reg 2.500000e+04 train accuracy: 0.318286 val accuracy: 0.33
7000
lr 2.000000e-07 reg 5.000000e+04 train accuracy: 0.303673 val accuracy: 0.32
8000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.357143 val accuracy: 0.36
3000
lr 5.000000e-07 reg 2.500000e+04 train accuracy: 0.326571 val accuracy: 0.34
4000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.313082 val accuracy: 0.32
6000
best validation accuracy achieved during cross-validation: 0.372000
```

```
In [19]: # evaluate on test set
# Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('softmax on raw pixels final test set accuracy: %f' % (test_accuracy,
softmax on raw pixels final test set accuracy: 0.358000
```

Inline Question - True or False

It's possible to add a new datapoint to a training set that would leave the SVM loss unchanged, but this is not the case with the Softmax classifier loss.

Answer: **True**

Explanation: For the SVM, $L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$ is exactly 0 whenever the correct score beats every other score by at least 1, i.e. $1(J) = 0$ for all j . So we can add a new point that the current W already classifies correctly with that margin; it contributes 0 to the summed data loss and 0 to the gradient.

For softmax, $L_i = -\log\left(\frac{e^{z_{y_i}}}{\sum_{j=1}^K e^{z_j}}\right)$. Because every $e^{z_j} > 0$, the probability

$p(y = y_i | x) < 1$ for any finite scores, so $L_i > 0$ always. Every new point therefore adds a strictly positive amount to the softmax loss.

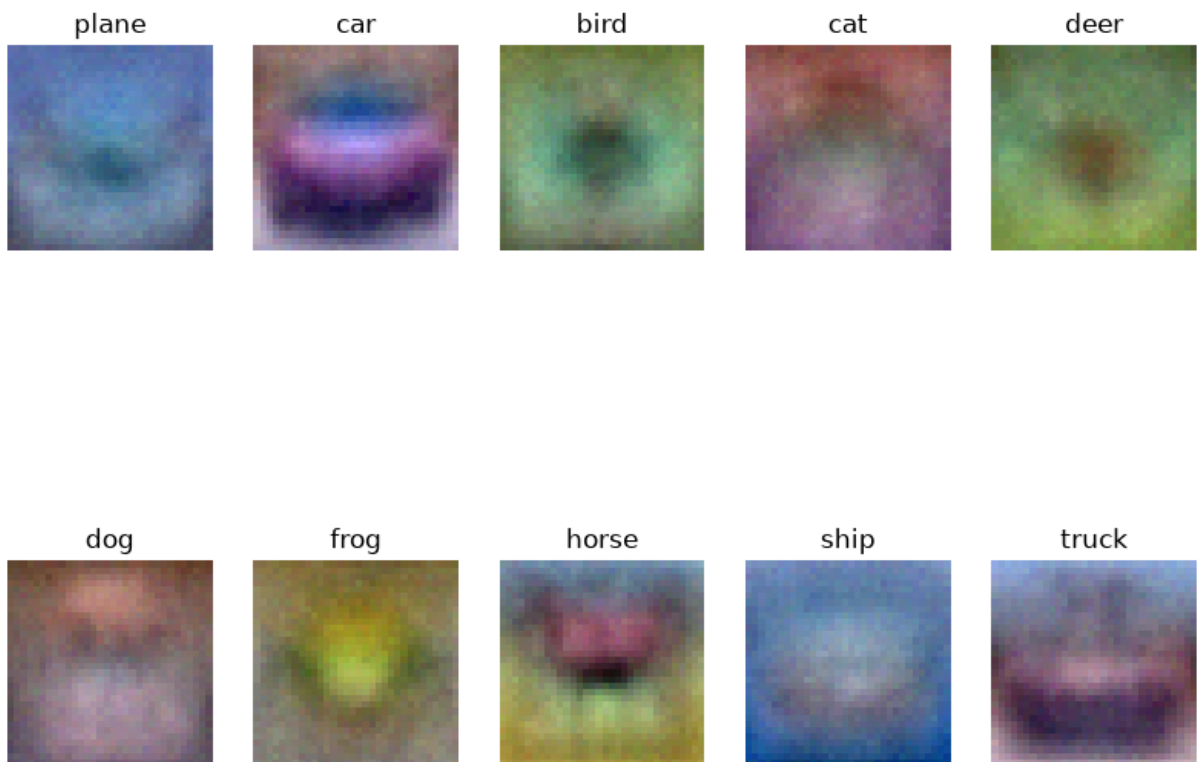
(Strictly, since $L = \frac{1}{N} \sum_i L_i$ is an average, adding a zero-loss point rescales the SVM average by $\frac{N}{N+1}$. The key idea is that the new point itself contributes zero hinge loss but always a positive softmax loss.)

```
In [20]: # Visualize the learned weights for each class
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)

w_min, w_max = np.min(w), np.max(w)

classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```



Implementing a Neural Network

In this exercise we will develop a neural network with fully-connected layers to perform classification, and test it out on the CIFAR-10 dataset.

```
In [16]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [17]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK FIRST ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [18]: # A bit of setup

from __future__ import print_function
import numpy as np
import matplotlib.pyplot as plt

from cs682.classifiers.neural_net import TwoLayerNet

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

We will use the class `TwoLayerNet` in the file

`cs682/classifiers/neural_net.py` to represent instances of our network. The network parameters are stored in the instance variable `self.params` where keys are string parameter names and values are numpy arrays. Below, we initialize toy data and a toy model that we will use to develop your implementation.

```
In [19]: # Create a small net and some toy data to check your implementations.
# Note that we set the random seed for repeatable experiments.

input_size = 4
hidden_size = 10
num_classes = 3
num_inputs = 5

def init_toy_model():
    np.random.seed(0)
    return TwoLayerNet(input_size, hidden_size, num_classes, std=1e-1)

def init_toy_data():
    np.random.seed(1)
    X = 10 * np.random.randn(num_inputs, input_size)
    y = np.array([0, 1, 2, 2, 1])
    return X, y

net = init_toy_model()
X, y = init_toy_data()
```

Forward pass: compute scores

Open the file `cs682/classifiers/neural_net.py` and look at the method `TwoLayerNet.loss`. This function is very similar to the loss functions you have written for the SVM and Softmax exercises: It takes the data and weights and computes the class scores, the loss, and the gradients on the parameters.

Implement the first part of the forward pass which uses the weights and biases to compute the scores for all inputs.

```
In [20]: scores = net.loss(X)
print('Your scores:')
print(scores)
print()
print('correct scores:')
correct_scores = np.asarray([
    [-0.81233741, -1.27654624, -0.70335995],
    [-0.17129677, -1.18803311, -0.47310444],
    [-0.51590475, -1.01354314, -0.8504215 ],
    [-0.15419291, -0.48629638, -0.52901952],
    [-0.00618733, -0.12435261, -0.15226949]])
print(correct_scores)
print()

# The difference should be very small. We get < 1e-7
print('Difference between your scores and correct scores:')
print(np.sum(np.abs(scores - correct_scores)))
```

Your scores:

```
[[-0.81233741 -1.27654624 -0.70335995]
 [-0.17129677 -1.18803311 -0.47310444]
 [-0.51590475 -1.01354314 -0.8504215 ]
 [-0.15419291 -0.48629638 -0.52901952]
 [-0.00618733 -0.12435261 -0.15226949]]
```

correct scores:

```
[[-0.81233741 -1.27654624 -0.70335995]
 [-0.17129677 -1.18803311 -0.47310444]
 [-0.51590475 -1.01354314 -0.8504215 ]
 [-0.15419291 -0.48629638 -0.52901952]
 [-0.00618733 -0.12435261 -0.15226949]]
```

Difference between your scores and correct scores:

3.680272120387684e-08

Forward pass: compute loss

In the same function, implement the second part that computes the data and regularization loss.

```
In [21]: loss, _ = net.loss(X, y, reg=0.05)
correct_loss = 1.30378789133

# should be very small, we get < 1e-12
print('Difference between your loss and correct loss:')
print(np.sum(np.abs(loss - correct_loss)))
```

Difference between your loss and correct loss:

1.7985612998927536e-13

Backward pass

Implement the rest of the function. This will compute the gradient of the loss with respect to the variables `W1`, `b1`, `W2`, and `b2`. Now that you (hopefully!) have a correctly implemented forward pass, you can debug your backward pass using a numeric gradient check:

```
In [22]: from cs682.gradient_check import eval_numerical_gradient

# Use numeric gradient checking to check your implementation of the backward
# If your implementation is correct, the difference between the numeric and
# analytic gradients should be less than 1e-8 for each of W1, W2, b1, and b2

loss, grads = net.loss(X, y, reg=0.05)

# these should all be less than 1e-8 or so
for param_name in grads:
    f = lambda W: net.loss(X, y, reg=0.05)[0]
```

```
param_grad_num = eval_numerical_gradient(f, net.params[param_name], verbose=1)
print('%s max relative error: %e' % (param_name, rel_error(param_grad_num)))
```

```
W2 max relative error: 3.440708e-09
b2 max relative error: 3.865112e-11
W1 max relative error: 3.561318e-09
b1 max relative error: 2.738420e-09
```

Train the network

To train the network we will use stochastic gradient descent (SGD), similar to the SVM and Softmax classifiers. Look at the function `TwoLayerNet.train` and fill in the missing sections to implement the training procedure. This should be very similar to the training procedure you used for the SVM and Softmax classifiers. You will also have to implement `TwoLayerNet.predict`, as the training process periodically performs prediction to keep track of accuracy over time while the network trains.

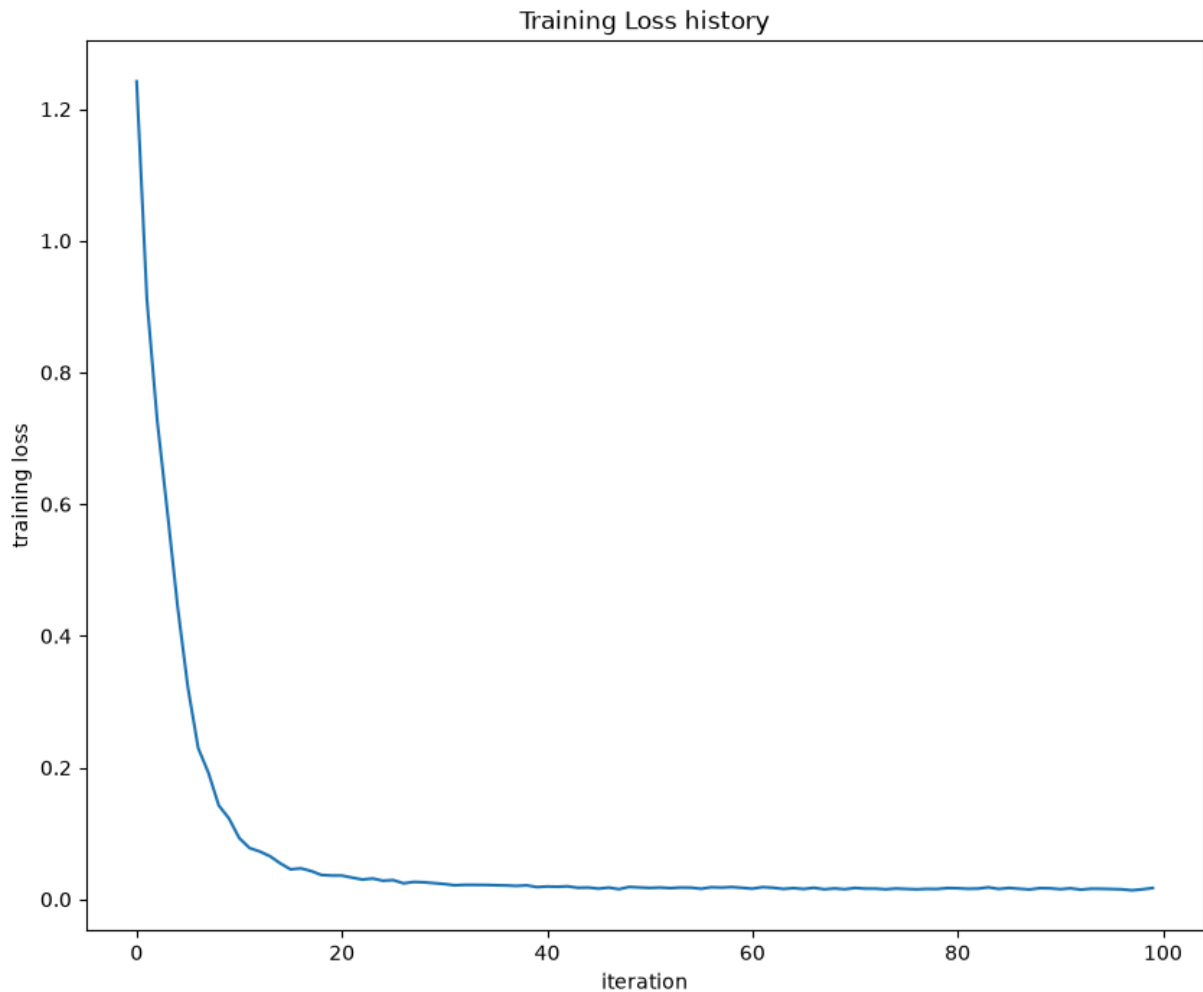
Once you have implemented the method, run the code below to train a two-layer network on toy data. You should achieve a training loss less than 0.2.

```
In [23]: net = init_toy_model()
stats = net.train(X, y, X, y,
                  learning_rate=1e-1, reg=5e-6,
                  num_iters=100, verbose=False)

print('Final training loss: ', stats['loss_history'][-1])

# plot the loss history
plt.plot(stats['loss_history'])
plt.xlabel('iteration')
plt.ylabel('training loss')
plt.title('Training Loss history')
plt.show()
```

Final training loss: 0.017149607938732093



Load the data

Now that you have implemented a two-layer network that passes gradient checks and works on toy data, it's time to load up our favorite CIFAR-10 data so we can use it to train a classifier on a real dataset.

```
In [24]: from cs682.data_utils import load_CIFAR10

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000)
    """
    Load the CIFAR-10 dataset from disk and perform preprocessing to prepare
    it for the two-layer neural net classifier. These are the same steps as
    we used for the SVM, but condensed to a single function.
    """
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs682/datasets/cifar-10-batches-py'

    # Don't forget to run get_datasets.sh, or this will throw an error
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
```

```

X_val = X_train[mask]
y_val = y_train[mask]
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Normalize the data: subtract the mean image
mean_image = np.mean(X_train, axis=0)
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image

# Reshape data to rows
X_train = X_train.reshape(num_training, -1)
X_val = X_val.reshape(num_validation, -1)
X_test = X_test.reshape(num_test, -1)

return X_train, y_train, X_val, y_val, X_test, y_test

# Cleaning up variables to prevent loading data multiple times (which may ca
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

# Invoke the above function to get our data.
X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

```

```

Clear previously loaded data.
Train data shape: (49000, 3072)
Train labels shape: (49000,)
Validation data shape: (1000, 3072)
Validation labels shape: (1000,)
Test data shape: (1000, 3072)
Test labels shape: (1000,)

```

Train a network

To train our network we will use SGD. In addition, we will adjust the learning rate with an exponential learning rate schedule as optimization proceeds; after each epoch, we will reduce the learning rate by multiplying it by a decay rate.

```
In [25]: input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
net = TwoLayerNet(input_size, hidden_size, num_classes)

# Train the network
stats = net.train(X_train, y_train, X_val, y_val,
                  num_iters=1000, batch_size=200,
                  learning_rate=1e-4, learning_rate_decay=0.95,
                  reg=0.25, verbose=True)

# Predict on the validation set
val_acc = (net.predict(X_val) == y_val).mean()
print('Validation accuracy: ', val_acc)
```

```
iteration 0 / 1000: loss 2.302954
iteration 800 / 1000: loss 2.006591
iteration 100 / 1000: loss 2.302550
iteration 200 / 1000: loss 2.297648
iteration 300 / 1000: loss 2.259602
iteration 400 / 1000: loss 2.204170
iteration 500 / 1000: loss 2.118565
iteration 600 / 1000: loss 2.051535
iteration 700 / 1000: loss 1.988466
iteration 900 / 1000: loss 1.951473
Validation accuracy: 0.287
```

Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.29 on the validation set. This isn't very good.

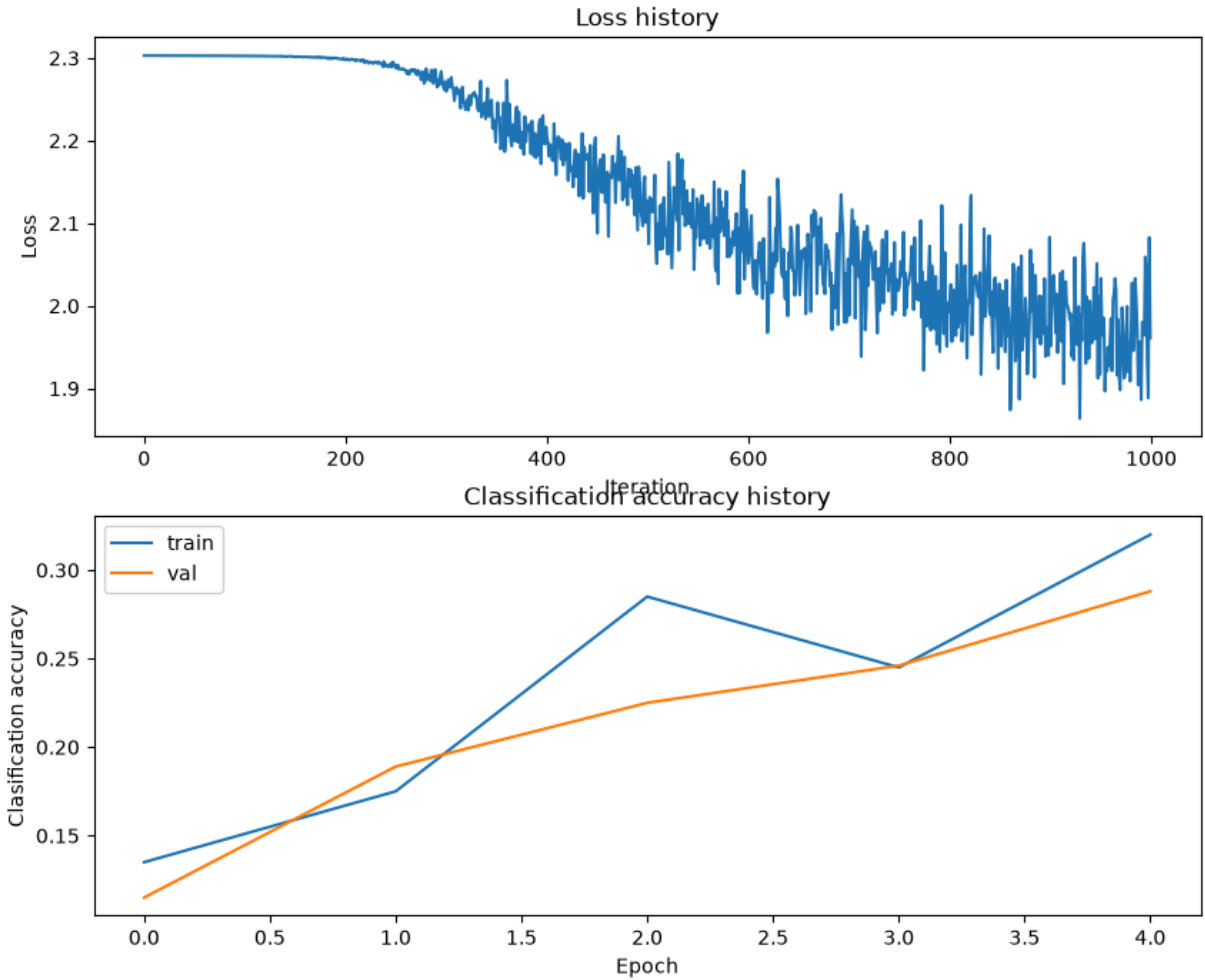
One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

```
In [26]: # Plot the loss function and train / validation accuracies
plt.subplot(2, 1, 1)
plt.plot(stats['loss_history'])
plt.title('Loss history')
plt.xlabel('Iteration')
plt.ylabel('Loss')

plt.subplot(2, 1, 2)
plt.plot(stats['train_acc_history'], label='train')
plt.plot(stats['val_acc_history'], label='val')
plt.title('Classification accuracy history')
plt.xlabel('Epoch')
plt.ylabel('Classification accuracy')
```

```
plt.legend()
plt.show()
```

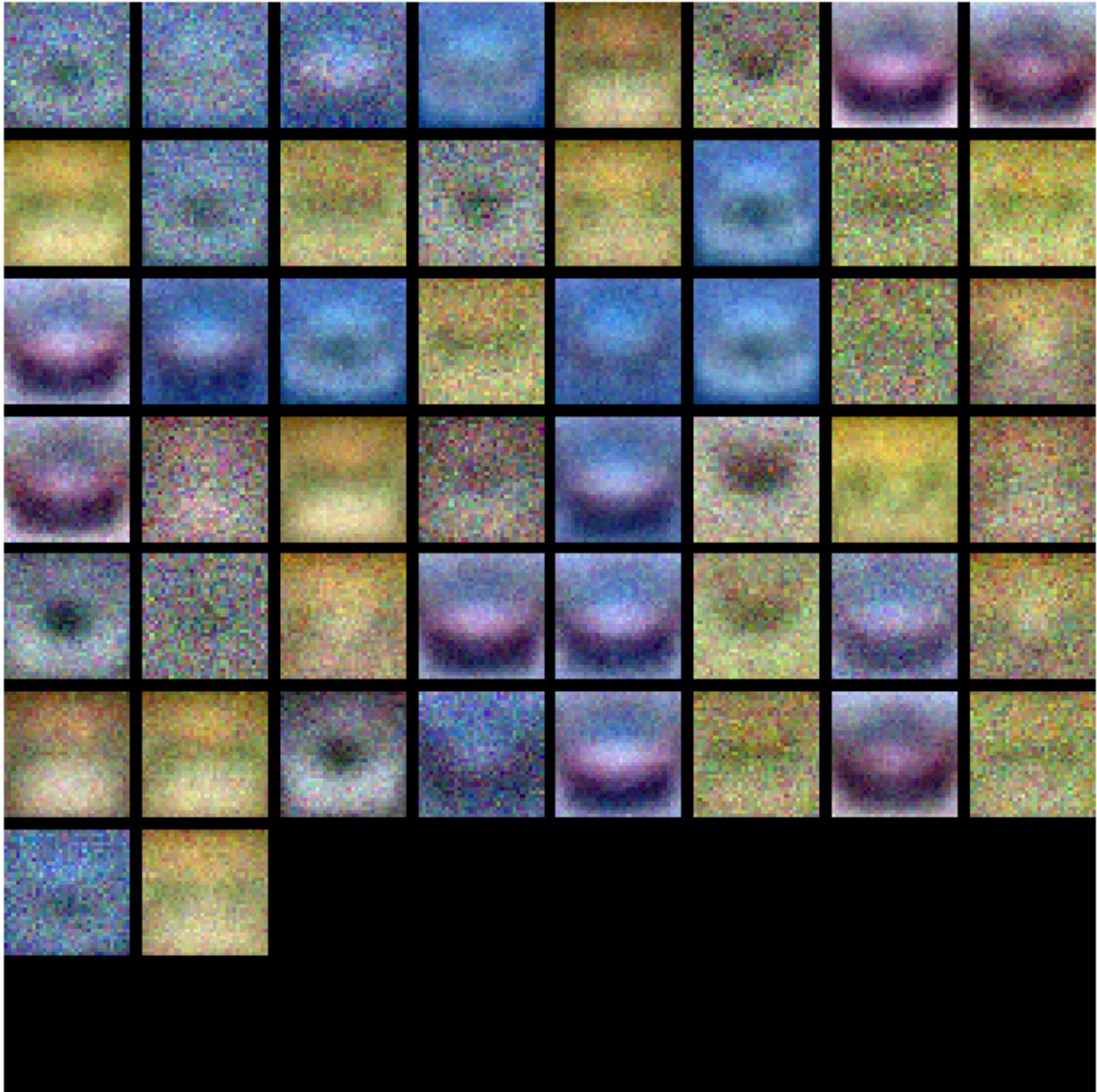


```
In [27]: from cs682.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(32, 32, 3, -1).transpose(3, 0, 1, 2)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(net)
```



Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, number of training epochs,

and regularization strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: Your goal in this exercise is to get as good of a result on CIFAR-10 as you can, with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

```
In [28]: best_net = None # store the best model into this

#####
# TODO: Tune hyperparameters using the validation set. Store your best train
# model in best_net.
#
# To help debug your network, it may help to use visualizations similar to t
# ones we used above; these visualizations will have significant qualitative
# differences from the ones we saw above for the poorly tuned network.
#
# Tweaking hyperparameters by hand can be fun, but you might find it useful
# write code to sweep through possible combinations of hyperparameters
# automatically like we did on the previous exercises.
#####
# The default run underfits: the loss falls almost linearly (learning rate t
# low), train and val accuracy are nearly equal (model has spare capacity),
# 1000 iterations is only ~4 epochs. So: raise the learning rate, train long
# use a bigger hidden layer and tune regularization on the validation set.
input_size = 32 * 32 * 3
num_classes = 10
best_val = -1
results = {}

hidden_sizes = [100, 150]
learning_rates = [1e-3, 1.5e-3]
regs = [0.1, 0.25]

for hs in hidden_sizes:
    for lr in learning_rates:
        for reg in regs:
            net = TwoLayerNet(input_size, hs, num_classes)
            stats = net.train(X_train, y_train, X_val, y_val,
                              num_iters=3000, batch_size=200,
                              learning_rate=lr, learning_rate_decay=0.95,
                              reg=reg, verbose=False)
            train_acc = (net.predict(X_train) == y_train).mean()
            val_acc = (net.predict(X_val) == y_val).mean()
            results[(hs, lr, reg)] = (train_acc, val_acc)
            print('hidden %d lr %.1e reg %.2f -> train %.3f val %.3f'
                  % (hs, lr, reg, train_acc, val_acc))
            if val_acc > best_val:
                best_val = val_acc
```

```

        best_net = net
        best_stats = stats

print('best validation accuracy: %f' % best_val)

# Plot the curves of the best model (should look much healthier than before)
plt.subplot(2, 1, 1)
plt.plot(best_stats['loss_history'])
plt.title('Loss history (best net)')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.subplot(2, 1, 2)
plt.plot(best_stats['train_acc_history'], label='train')
plt.plot(best_stats['val_acc_history'], label='val')
plt.title('Classification accuracy history (best net)')
plt.xlabel('Epoch')
plt.ylabel('Classification accuracy')
plt.legend()
plt.show()

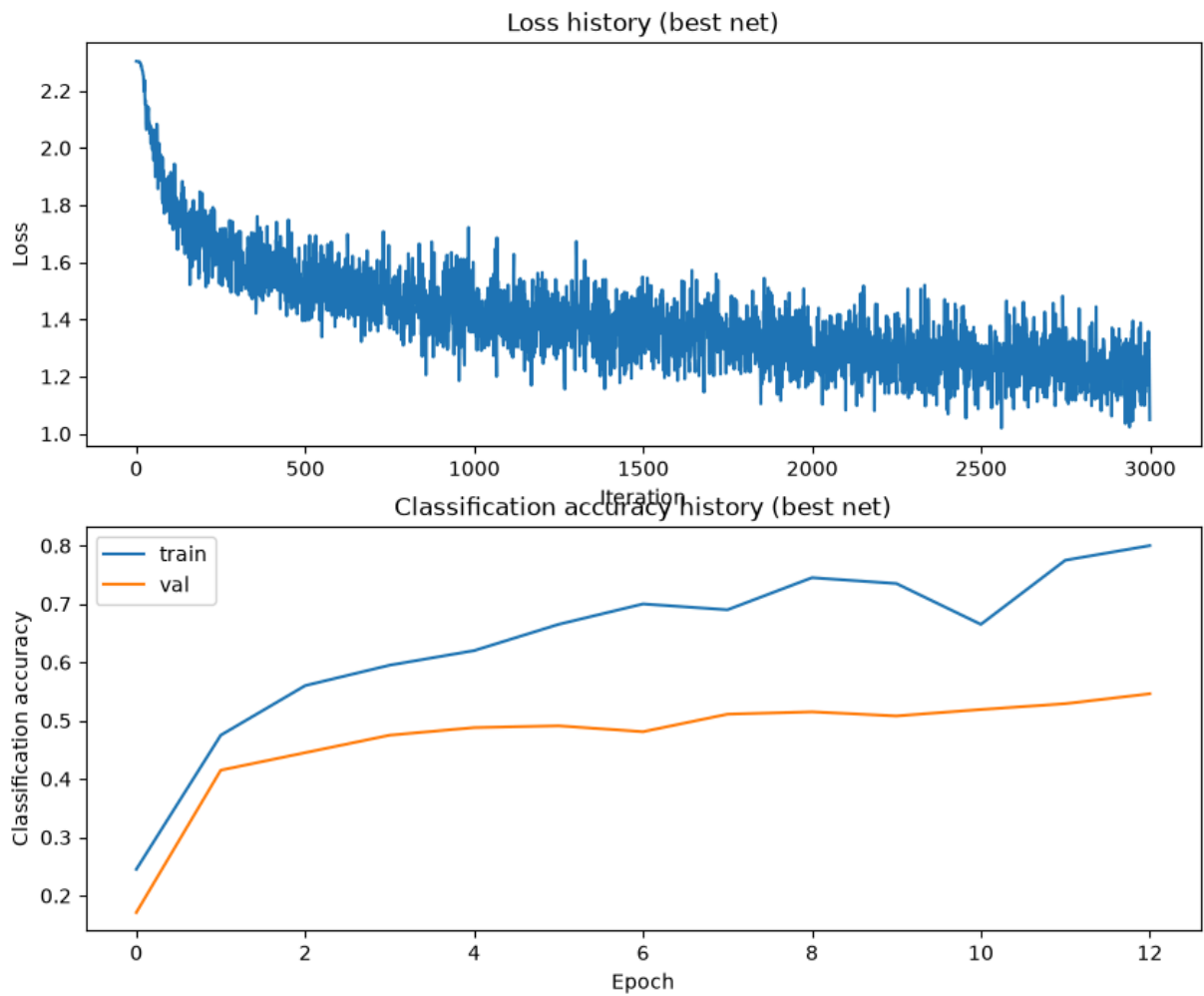
#####
#                                     END OF YOUR CODE
#####

```

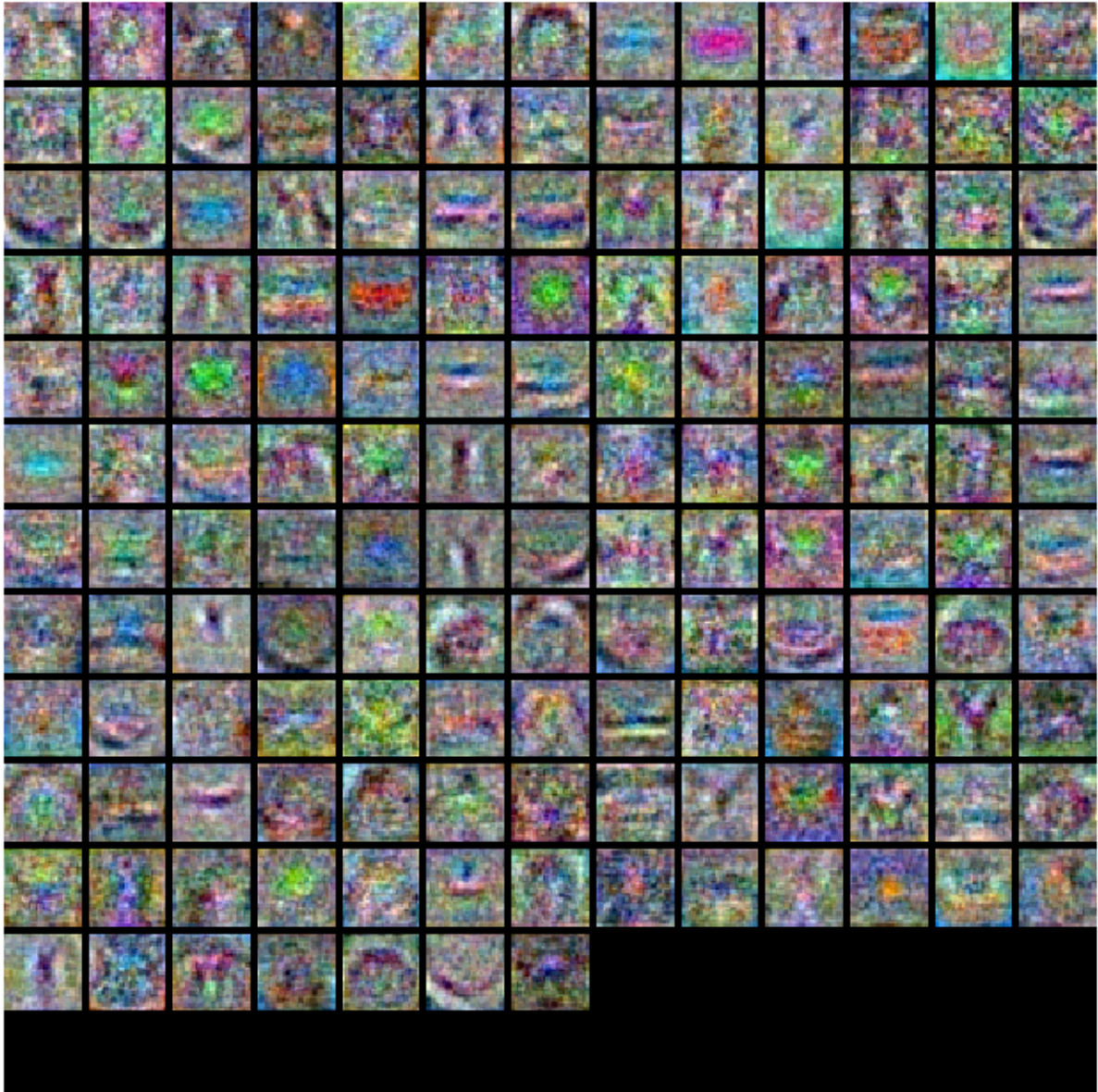
```

hidden 100 lr 1.0e-03 reg 0.10 -> train 0.583 val 0.523
hidden 100 lr 1.0e-03 reg 0.25 -> train 0.574 val 0.526
hidden 100 lr 1.5e-03 reg 0.10 -> train 0.603 val 0.517
hidden 100 lr 1.5e-03 reg 0.25 -> train 0.578 val 0.503
hidden 150 lr 1.0e-03 reg 0.10 -> train 0.606 val 0.533
hidden 150 lr 1.0e-03 reg 0.25 -> train 0.589 val 0.510
hidden 150 lr 1.5e-03 reg 0.10 -> train 0.618 val 0.536
hidden 150 lr 1.5e-03 reg 0.25 -> train 0.596 val 0.527
best validation accuracy: 0.536000

```



```
In [29]: # visualize the weights of the best network  
show_net_weights(best_net)
```



Run on the test set

When you are done experimenting, you should evaluate your final trained network on the test set; you should get above 48%.

```
In [30]: test_acc = (best_net.predict(X_test) == y_test).mean()  
print('Test accuracy: ', test_acc)
```

Test accuracy: 0.546

Inline Question

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

Answer: 1, 3

Explanation: The train/test gap is overfitting (high variance).

1. **Training on a larger dataset** helps: with more (and more varied) data the network cannot simply memorise the training set, so it generalises better and the gap shrinks.
2. **Adding more hidden units does not** help: it increases model capacity, which makes it easier to fit the training set and usually *widens* the gap (unless it is paired with stronger regularization).
3. **Increasing regularization strength** helps: a larger L2 penalty keeps the weights small and the function smoother, lowering training accuracy a bit but reducing overfitting, so the gap narrows (too much will underfit).

Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

```
In [23]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [24]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK FIRST ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [25]: from __future__ import print_function
import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

```
In [26]: from cs682.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000)
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs682/datasets/cifar-10-batches-py'

    # Don't forget to run get_datasets.sh, or this will throw an error
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

# Cleaning up variables to prevent loading data multiple times (which may ca
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
```

Clear previously loaded data.

Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your interests.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each

image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

```
In [27]: from cs682.features import *

num_color_bins = 10 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img, nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images

Train SVM on features

Using the multiclass SVM code developed earlier in the assignment, train SVMs on top of the features extracted above; this should achieve better results than training SVMs directly on top of raw pixels.

```
In [28]: # Use the validation set to tune the learning rate and regularization strength

from cs682.classifiers.linear_classifier import LinearSVM

learning_rates = [1e-9, 1e-8, 1e-7]
regularization_strengths = [5e4, 5e5, 5e6]

results = {}
best_val = -1
best_svm = None

#####
# TODO:
# Use the validation set to set the learning rate and regularization strength
# This should be identical to the validation that you did for the SVM; save
# the best trained classifier in best_svm. You might also want to play
# with different numbers of bins in the color histogram. If you are careful
# you should be able to get accuracy of near 0.44 on the validation set.
#####
# The provided values over-regularize these standardized 154-d features (even
# pair with lr * reg = 5e-3 collapses W to the same class-mean template, ~0.
# Larger learning rates with much smaller regularization are searched as well
# Pairs with lr * reg >= 1 are skipped because training diverges.
search_lrs = learning_rates + [1e-3, 3e-3, 1e-2, 3e-2]
search_regs = regularization_strengths + [0, 1e-3, 1e-2, 1e-1]

for lr in search_lrs:
    for reg in search_regs:
        if lr * reg >= 1:
            continue
        svm = LinearSVM()
        svm.train(X_train_feats, y_train, learning_rate=lr, reg=reg,
                  num_iters=3000, verbose=False)
        train_accuracy = np.mean(svm.predict(X_train_feats) == y_train)
        val_accuracy = np.mean(svm.predict(X_val_feats) == y_val)
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_svm = svm

#####
#                                     END OF YOUR CODE
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
```

```
lr, reg, train_accuracy, val_accuracy))  
print('best validation accuracy achieved during cross-validation: %f' % best
```

```
lr 1.000000e-09 reg 0.000000e+00 train accuracy: 0.122388 val accuracy: 0.113000
lr 1.000000e-09 reg 1.000000e-03 train accuracy: 0.084082 val accuracy: 0.086000
lr 1.000000e-09 reg 1.000000e-02 train accuracy: 0.126490 val accuracy: 0.109000
lr 1.000000e-09 reg 1.000000e-01 train accuracy: 0.090061 val accuracy: 0.090000
lr 1.000000e-09 reg 5.000000e+04 train accuracy: 0.105102 val accuracy: 0.106000
lr 1.000000e-09 reg 5.000000e+05 train accuracy: 0.122857 val accuracy: 0.110000
lr 1.000000e-09 reg 5.000000e+06 train accuracy: 0.416857 val accuracy: 0.409000
lr 1.000000e-08 reg 0.000000e+00 train accuracy: 0.095653 val accuracy: 0.101000
lr 1.000000e-08 reg 1.000000e-03 train accuracy: 0.137184 val accuracy: 0.131000
lr 1.000000e-08 reg 1.000000e-02 train accuracy: 0.104102 val accuracy: 0.113000
lr 1.000000e-08 reg 1.000000e-01 train accuracy: 0.113347 val accuracy: 0.106000
lr 1.000000e-08 reg 5.000000e+04 train accuracy: 0.123204 val accuracy: 0.107000
lr 1.000000e-08 reg 5.000000e+05 train accuracy: 0.415898 val accuracy: 0.417000
lr 1.000000e-08 reg 5.000000e+06 train accuracy: 0.401714 val accuracy: 0.401000
lr 1.000000e-07 reg 0.000000e+00 train accuracy: 0.154592 val accuracy: 0.145000
lr 1.000000e-07 reg 1.000000e-03 train accuracy: 0.153000 val accuracy: 0.135000
lr 1.000000e-07 reg 1.000000e-02 train accuracy: 0.142816 val accuracy: 0.140000
lr 1.000000e-07 reg 1.000000e-01 train accuracy: 0.126429 val accuracy: 0.145000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.416776 val accuracy: 0.417000
lr 1.000000e-07 reg 5.000000e+05 train accuracy: 0.404857 val accuracy: 0.399000
lr 1.000000e-07 reg 5.000000e+06 train accuracy: 0.318980 val accuracy: 0.307000
lr 1.000000e-03 reg 0.000000e+00 train accuracy: 0.507367 val accuracy: 0.496000
lr 1.000000e-03 reg 1.000000e-03 train accuracy: 0.508449 val accuracy: 0.495000
lr 1.000000e-03 reg 1.000000e-02 train accuracy: 0.508347 val accuracy: 0.497000
lr 1.000000e-03 reg 1.000000e-01 train accuracy: 0.505102 val accuracy: 0.494000
lr 3.000000e-03 reg 0.000000e+00 train accuracy: 0.511388 val accuracy: 0.502000
lr 3.000000e-03 reg 1.000000e-03 train accuracy: 0.511265 val accuracy: 0.506000
lr 3.000000e-03 reg 1.000000e-02 train accuracy: 0.514082 val accuracy: 0.498000
```

```

lr 3.000000e-03 reg 1.000000e-01 train accuracy: 0.507163 val accuracy: 0.486000
lr 1.000000e-02 reg 0.000000e+00 train accuracy: 0.509082 val accuracy: 0.489000
lr 1.000000e-02 reg 1.000000e-03 train accuracy: 0.511204 val accuracy: 0.496000
lr 1.000000e-02 reg 1.000000e-02 train accuracy: 0.508694 val accuracy: 0.494000
lr 1.000000e-02 reg 1.000000e-01 train accuracy: 0.501143 val accuracy: 0.494000
lr 3.000000e-02 reg 0.000000e+00 train accuracy: 0.503020 val accuracy: 0.493000
lr 3.000000e-02 reg 1.000000e-03 train accuracy: 0.501959 val accuracy: 0.483000
lr 3.000000e-02 reg 1.000000e-02 train accuracy: 0.507020 val accuracy: 0.498000
lr 3.000000e-02 reg 1.000000e-01 train accuracy: 0.495245 val accuracy: 0.496000
best validation accuracy achieved during cross-validation: 0.506000

```

```

In [29]: # Evaluate your trained SVM on the test set
y_test_pred = best_svm.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)

```

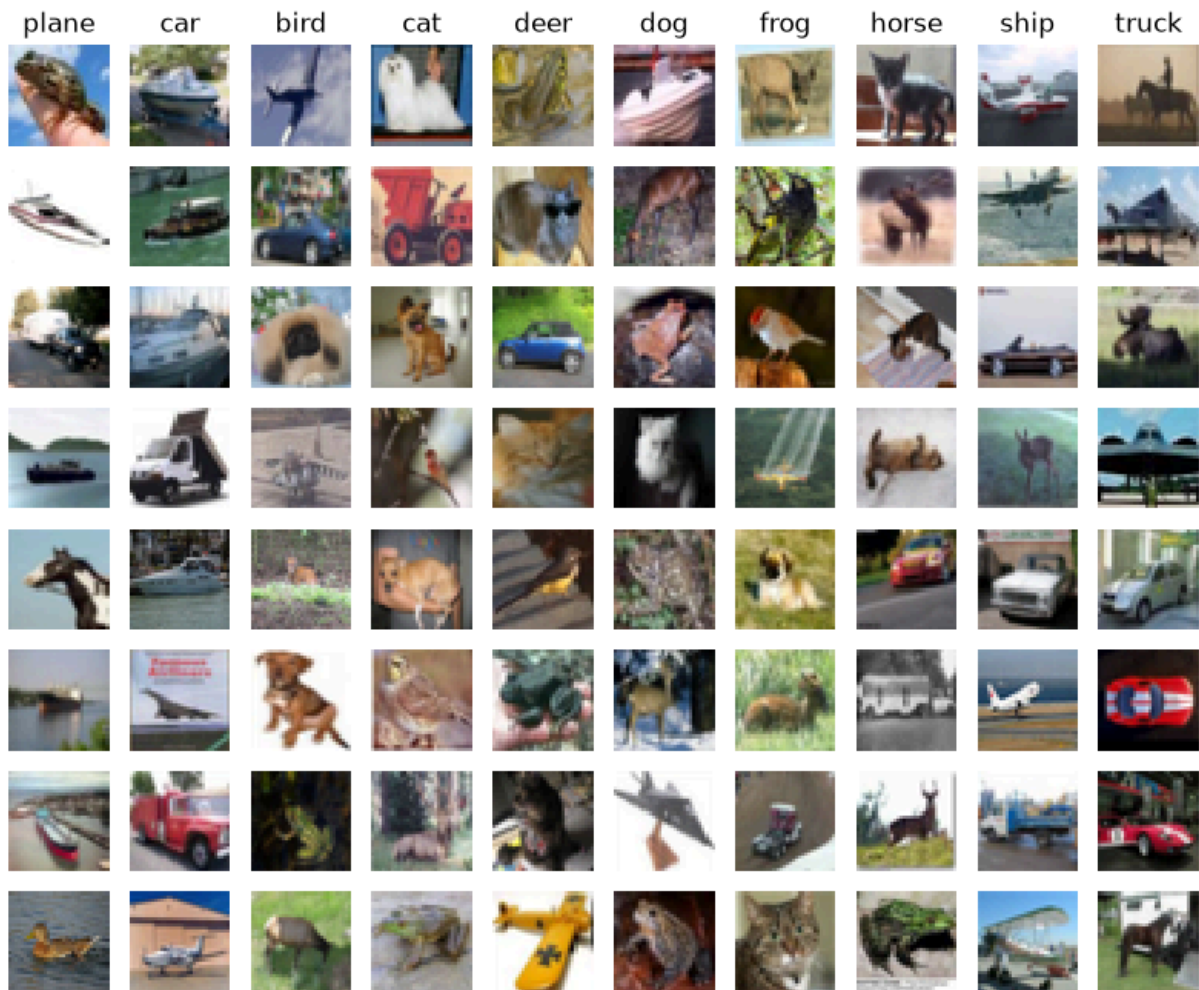
0.492

```

In [30]: # An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls_name)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()

```



Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Answer: Many of the mistakes make sense given what the features capture. HOG describes coarse edge orientations (shape and outline) and the hue histogram describes the overall color distribution, not what the object actually is. In my results:

- *Plane* and *ship* are confused both ways: the *plane* column is mostly boats and ships, and the *ship* column is mostly planes. Both usually sit on a plain blue or grey background (sky or water) that dominates the color histogram, and both have long horizontal outlines.
- Vehicles are confused with each other: the *car* column contains several boats, trucks (including a dump truck and a red fire truck) and planes, and the *truck* column contains several cars. They share boxy, straight-edged shapes.
- Animals are confused with each other: the *cat* column is mostly dogs, the *horse* column contains cats and dogs, and the *bird*, *deer*, *dog* and *frog* columns mix deer, cats, dogs, frogs and birds. They have similar outlines and brown or green natural backgrounds.

- Some errors don't make intuitive sense, such as horses (one with a rider) labeled *truck*, a yellow plane labeled *deer*, or a red truck labeled *cat*. These images have dominant edges or colors that happen to match another class's template, which a linear classifier on hand-crafted features can't separate.

Overall, the misclassifications mostly reflect visual similarity at the level of these features, which is why the neural network below does better.

Neural Network on image features

Earlier in this assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

```
In [31]: # Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

```
(49000, 155)
```

```
(49000, 154)
```

```
In [32]: from cs682.classifiers.neural_net import TwoLayerNet

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

net = TwoLayerNet(input_dim, hidden_dim, num_classes)
best_net = None

#####
# TODO: Train a two-layer neural network on image features. You may want to
# cross-validate various parameters as in previous sections. Store your best
# model in the best_net variable.
#####
# Feature vectors are small (~154-d) and normalised, so much larger learning
# rates than on raw pixels are appropriate.
best_val = -1
results = {}
learning_rates = [1e-1, 3e-1, 5e-1]
```

```

regs = [1e-4, 1e-3, 5e-3]

for lr in learning_rates:
    for reg in regs:
        net = TwoLayerNet(input_dim, hidden_dim, num_classes)
        net.train(X_train_feats, y_train, X_val_feats, y_val,
                  num_iters=2000, batch_size=200,
                  learning_rate=lr, learning_rate_decay=0.95,
                  reg=reg, verbose=False)
        train_acc = (net.predict(X_train_feats) == y_train).mean()
        val_acc = (net.predict(X_val_feats) == y_val).mean()
        results[(lr, reg)] = (train_acc, val_acc)
        print('lr %.1e reg %.1e -> train %.3f val %.3f' % (lr, reg, train_acc, val_acc))
        if val_acc > best_val:
            best_val = val_acc
            best_net = net

print('best validation accuracy: %f' % best_val)
#####
#                               END OF YOUR CODE
#####

```

```

lr 1.0e-01 reg 1.0e-04 -> train 0.558 val 0.545
lr 1.0e-01 reg 1.0e-03 -> train 0.555 val 0.534
lr 1.0e-01 reg 5.0e-03 -> train 0.535 val 0.521
lr 3.0e-01 reg 1.0e-04 -> train 0.670 val 0.583
lr 3.0e-01 reg 1.0e-03 -> train 0.646 val 0.602
lr 3.0e-01 reg 5.0e-03 -> train 0.566 val 0.540
lr 5.0e-01 reg 1.0e-04 -> train 0.724 val 0.594
lr 5.0e-01 reg 1.0e-03 -> train 0.672 val 0.577
lr 5.0e-01 reg 5.0e-03 -> train 0.574 val 0.555
best validation accuracy: 0.602000

```

In [33]: *# Run your best neural net classifier on the test set. You should be able to get more than 55% accuracy.*

```

test_acc = (best_net.predict(X_test_feats) == y_test).mean()
print(test_acc)

```

0.57